

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»**

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

«Запросы на выборку и модификацию данных. Представления.

Работа с индексами»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Журбина Марина Андреевна

Факультет Прикладной информатики

Группа К3239

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2025/2026

Цель работы:

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание:

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

Схема логической модели базы данных ERD:

<https://drive.google.com/file/d/10D2sghuRWunJyvJHy8q0UajLmZZjt62x/view?usp=sharing>

Выполнение:

Наименование БД: Courses.

Вариант: 7.

1. Запросы к базе данных согласно индивидуальному заданию, ч. 2:

Формулировка запроса № 1: Вывести все номера групп и программы, где количество слушателей меньше 10.

SQL-команда запроса № 1:

```
SET search_path TO lr3;  
  
SELECT g.group_number, p.program_name, COUNT(t.trainee_id) as student_count
```

```

FROM training_group g
JOIN educational_program p ON g.program_id = p.program_id
LEFT JOIN trainee t ON g.group_id = t.group_id
GROUP BY g.group_number, p.program_name
HAVING COUNT(t.trainee_id) < 10;

```

Результат выполнения запроса № 1:

	group_number character varying (10)	program_name character varying (200)	student_count bigint
1	СА-24	Системное администрирован...	4
2	ПИТ-2023	Основы Python	1
3	БД-2023	Разработчик БД	1

Формулировка запроса № 2: Вывести список преподавателей с указанием количества программ, где они преподавали за истекший учебный год.

SQL-команда запроса № 2:

```

SET search_path TO lr3;

SELECT t.last_name, t.first_name, COUNT(DISTINCT g.program_id) as
programs_count
FROM teacher t
JOIN lesson l ON t.teacher_id = l.teacher_id
JOIN training_group g ON l.group_id = g.group_id
WHERE l.lesson_date >= CURRENT_DATE - INTERVAL '1 year'
GROUP BY t.teacher_id, t.last_name, t.first_name;

```

Результат выполнения запроса № 2:

	last_name character varying (30)	first_name character varying (30)	programs_count bigint
1	Иванов	Иван	2
2	Петрова	Анна	2

Формулировка запроса № 3: Вывести список преподавателей, которые не проводят занятия на третьей паре ни в один из дней недели.

SQL-команда запроса № 3:

```

SET search_path TO lr3;

SELECT teacher_id, last_name, first_name

```

```

FROM teacher
WHERE teacher_id NOT IN (
    SELECT DISTINCT teacher_id
    FROM lesson
    WHERE lesson_time = '14:00:00'
);

```

Результат выполнения запроса № 3:

	teacher_id [PK] integer	last_name character varying (30)	first_name character varying (30)
1	3	Волков	Дмитрий
2	4	Зайцева	Екатерина

Формулировка запроса № 4: Вывести список свободных лекционных аудиторий на ближайший понедельник.

SQL-команда запроса № 4:

```

SET search_path TO lr3;

SELECT r.room_number, rt.type_name
FROM room r
JOIN room_type rt ON r.room_type_id = rt.room_type_id
WHERE rt.type_name = 'Лекционная'
    AND r.room_id NOT IN (
        SELECT room_id
        FROM lesson
        WHERE lesson_date = date_trunc('week', CURRENT_DATE) + INTERVAL '1
week'
    );

```

Результат выполнения запроса № 4:

	room_number character varying (6)	type_name character varying (20)
1	501	Лекционная

Формулировка запроса № 5: Вычислить общее количество обучающихся по каждой программе за последний год.

SQL-команда запроса № 5:

```

SET search_path TO lr3;

```

```

SELECT p.program_name, COUNT(t.trainee_id) as total_students
FROM educational_program p
JOIN training_group g ON p.program_id = g.program_id
JOIN trainee t ON g.group_id = t.group_id
WHERE g.start_date <= CURRENT_DATE
      AND (g.end_date >= CURRENT_DATE - INTERVAL '1 year' OR g.end_date IS NULL)
GROUP BY p.program_name;

```

Результат выполнения запроса № 5:

	program_name character varying (200)	total_students bigint
1	Основы Python	1
2	Разработчик БД	1
3	Системное администрирован...	4

Формулировка запроса № 6: Вычислить среднюю загруженность компьютерных классов в неделю за последний месяц (в часах).

SQL-команда запроса № 6:

```

SET search_path TO lr3;

SELECT r.room_number,
       ROUND((COUNT(l.lesson_id) * 2.0 / 4.0), 2) as avg_weekly_hours
FROM room r
JOIN room_type rt ON r.room_type_id = rt.room_type_id
LEFT JOIN lesson l ON r.room_id = l.room_id
                AND l.lesson_date >= CURRENT_DATE - INTERVAL '1 month'
WHERE rt.type_name = 'Компьютерный класс'
GROUP BY r.room_number;

```

Результат выполнения запроса № 6:

	room_number character varying (6)	avg_weekly_hours numeric
1	205	2.50
2	502	0.00

Формулировка запроса № 7: Найти самые популярные программы за последние 3 года.

SQL-команда запроса № 7:

```
SET search_path TO lr3;

SELECT
    p.program_name,
    COUNT(t.trainee_id) AS students_count
FROM educational_program p
JOIN training_group g ON g.program_id = p.program_id
JOIN trainee t ON t.group_id = g.group_id
WHERE g.start_date <= CURRENT_DATE
    AND (g.end_date >= CURRENT_DATE - INTERVAL '3 years' OR g.end_date IS NULL)
GROUP BY p.program_name
HAVING COUNT(t.trainee_id) = (
    SELECT
        MAX(cnt)
    FROM (
        SELECT
            COUNT(t2.trainee_id) AS cnt
        FROM educational_program p2
        JOIN training_group g2 ON g2.program_id = p2.program_id
        JOIN trainee t2 ON t2.group_id = g2.group_id
        WHERE g2.start_date <= CURRENT_DATE
            AND (g2.end_date >= CURRENT_DATE - INTERVAL '3 years' OR
g2.end_date IS NULL)
        GROUP BY p2.program_id
    ) AS counts
)
ORDER BY p.program_name;
```

Результат выполнения запроса № 7:

	program_name character varying (200) 	students_count bigint 
1	Системное администрирован...	4

2. Запросы на создание представлений согласно индивидуальному заданию, ч. 3:

Формулировка запроса на создание представления № 1: Создать представление для потенциальных слушателей, содержащее перечень специальностей, изучаемых на них дисциплин и количество часов.

SQL-команда запроса на создание представления № 1:

```
CREATE OR REPLACE VIEW view_potential_students AS
SELECT p.program_name AS "Программа",
       d.discipline_name AS "Дисциплина",
       (d.lecture_hours + d.practice_hours + d.lab_hours) AS "Часы по
дисциплине"
FROM educational_program p
JOIN program_discipline pd ON p.program_id = pd.program_id
JOIN discipline d ON pd.discipline_id = d.discipline_id;
```

SQL-команда для просмотра содержимого представления № 1:

```
SELECT * FROM view_potential_students;
```

Содержимое представления № 1:

	Программа character varying (200)	Дисциплина character varying (200)	Часы по дисциплине integer
1	Разработчик БД	Базы данных	50
2	Разработчик БД	Программирование	50
3	Основы Python	Программирование	50
4	Основы Python	Базы данных	50
5	Системное администрирован...	Базы данных	50

Формулировка запроса на создание представления № 2: Создать представление общих доход по каждой программе за последний год.

SQL-команда запроса на создание представления № 2:

```
SET search_path TO lr3;

CREATE OR REPLACE VIEW view_program_income AS
SELECT
    p.program_name AS "Программа",
    COALESCE(COUNT(t.trainee_id), 0) * p.price AS "Общий доход (руб)"
FROM educational_program p
LEFT JOIN training_group g
    ON g.program_id = p.program_id
    AND g.start_date >= CURRENT_DATE - INTERVAL '1 year'
```

```

LEFT JOIN trainee t
  ON t.group_id = g.group_id
GROUP BY
  p.program_id,
  p.program_name,
  p.price;

```

SQL-команда для просмотра содержимого представления № 2:

```

SELECT
  "Программа",
  "Общий доход (руб)"
FROM view_program_income
ORDER BY
  "Общий доход (руб)" DESC,
  "Программа";

```

Содержимое представления № 2:

	Программа character varying (200)	Общий доход (руб) numeric
1	Системное администрирован...	120000.00
2	Разработчик БД	90000.00
3	Основы Python	0.00

3. Запросы на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов:

Формулировка запроса INSERT с подзапросом: Добавить нового слушателя в уже существующую заданную группу 'БД-2023'.

SQL-команда для просмотра всех слушателей конкретной группы:

```

SET search_path TO lr3;

SELECT
  g.group_number,
  t.trainee_id,
  t.last_name,
  t.first_name,
  t.middle_name,
  t.birth_date,
  t.phones,

```



```

t.email,
ts.status_name AS trainee_status
FROM training_group g
JOIN trainee t
  ON t.group_id = g.group_id
JOIN trainee_status ts
  ON ts.trainee_status_id = t.trainee_status_id
WHERE g.group_number = 'БД-2023'
ORDER BY t.last_name, t.first_name, t.middle_name;

```

Результат до запроса INSERT:

	group_number character varying (10)	trainee_id integer	last_name character varying (30)	first_name character varying (30)	middle_name character varying (30)	birth_date date	phones character varying (200)	email character varying (200)	trainee_status character varying (20)
1	БД-2023	1	Смирнов	Алексей	Николаевич	2006-05-10	89001234567	smirnov@mail.ru	Обучается

SQL-команда запроса INSERT:

```

INSERT INTO trainee (last_name, first_name, middle_name, birth_date,
group_id, phones, email, trainee_status_id)
VALUES (
'Орлова',
'Дарья',
'Ильинична',
'2002-10-12',
(SELECT group_id FROM training_group WHERE group_number = 'БД-2023'),
'89990001122',
'orlova@test.com',
(SELECT trainee_status_id FROM trainee_status WHERE status_name =
'Обучается')
);

```

Результат после выполнения запроса INSERT:

	group_number character varying (10)	trainee_id integer	last_name character varying (30)	first_name character varying (30)	middle_name character varying (30)	birth_date date	phones character varying (200)	email character varying (200)	trainee_status character varying (20)
1	БД-2023	7	Орлова	Дарья	Ильинична	2002-10-12	89990001122	orlova@test.com	Обучается
2	БД-2023	1	Смирнов	Алексей	Николаевич	2006-05-10	89001234567	smirnov@mail.ru	Обучается

Формулировка запроса UPDATE с подзапросом: Изменить статус на 'Уволен' у преподавателей, которые не вели занятия за последний год.

SQL-команда для просмотра всех преподавателей и количество их занятий за последний год:

```
SET search_path TO 1r3;
```

```
SELECT
```

```

tch.teacher_id,
tch.last_name,
ts.status_name,
COUNT(l.lesson_id) FILTER (
    WHERE l.lesson_date >= (CURRENT_DATE - INTERVAL '1 year')
) AS lessons_last_year
FROM teacher tch
JOIN teacher_status ts
    ON ts.teacher_status_id = tch.teacher_status_id
LEFT JOIN lesson l
    ON l.teacher_id = tch.teacher_id
GROUP BY tch.teacher_id, tch.last_name, ts.status_name
ORDER BY tch.teacher_id;

```

Результат до запроса UPDATE:

	teacher_id integer	last_name character varying (30)	status_name character varying (20)	lessons_last_year bigint
1	1	Иванов	В штате	3
2	2	Петрова	В штате	5
3	3	Волков	В штате	0
4	4	Зайцева	В штате	0

SQL-команда запроса UPDATE:

```

UPDATE teacher
SET teacher_status_id = (SELECT teacher_status_id FROM teacher_status WHERE
status_name = 'Уволен')
WHERE teacher_id IN (
    SELECT tch.teacher_id
    FROM teacher tch
    WHERE NOT EXISTS (
        SELECT 1
        FROM lesson l
        WHERE l.teacher_id = tch.teacher_id
        AND l.lesson_date >= (current_date - INTERVAL '1 year')));

```

Результат после выполнения запроса UPDATE:

	teacher_id integer	last_name character varying (30)	status_name character varying (20)	lessons_last_year bigint
1	1	Иванов	В штате	3
2	2	Петрова	В штате	5
3	3	Волков	Уволен	0
4	4	Зайцева	Уволен	0

Формулировка запроса DELETE с подзапросом: Удалить все занятия для групп, которые закончили обучение.

SQL-команда для просмотра всех занятий для групп, которые закончили обучение:

```
SET search_path TO lr3;

SELECT
    l.lesson_id, l.lesson_date, l.lesson_time,
    g.group_number, g.end_date
FROM lesson l
JOIN training_group g ON g.group_id = l.group_id
WHERE g.end_date IS NOT NULL
    AND g.end_date < CURRENT_DATE
ORDER BY l.lesson_date;
```

Результат до запроса DELETE:

	lesson_id integer	lesson_date date	lesson_time time without time zone	group_number character varying (10)	end_date date
1	34	2025-10-15	09:00:00	ПИТ-2023	2025-12-30
2	35	2025-10-20	11:30:00	ПИТ-2023	2025-12-30
3	8	2026-03-16	12:00:00	ПИТ-2023	2025-12-30
4	7	2026-03-21	10:00:00	ПИТ-2023	2025-12-30
5	6	2026-03-31	10:00:00	ПИТ-2023	2025-12-30
6	5	2026-04-05	14:00:00	ПИТ-2023	2025-12-30

SQL-команда запроса DELETE:

```
DELETE FROM lesson
WHERE lesson_id IN (
    SELECT l.lesson_id
    FROM lesson l
    JOIN training_group g ON g.group_id = l.group_id
    WHERE g.end_date IS NOT NULL
```

```
AND g.end_date < CURRENT_DATE  
);
```

Результат после выполнения запроса DELETE:

	lesson_id integer	lesson_date date	lesson_time time without time zone	group_number character varying (10)	end_date date
--	----------------------	---------------------	---------------------------------------	--	------------------

4. Создание индексов:

Формулировка запроса № 1 с простым индексом: Получить занятия за последний месяц.

SQL-команда запроса № 1:

```
EXPLAIN (ANALYZE, BUFFERS)  
SELECT *  
FROM lesson  
WHERE lesson_date >= CURRENT_DATE - INTERVAL '1 month';
```

Результат до создания простого индекса:

	QUERY PLAN text
1	Seq Scan on lesson (cost=0.00..2.23 rows=53 width=36) (actual time=0.029..0.043 rows=52.00 loop...
2	Filter: (lesson_date >= (CURRENT_DATE - '1 mon'::interval))
3	Rows Removed by Filter: 18
4	Buffers: shared hit=1
5	Planning Time: 0.135 ms
6	Execution Time: 0.064 ms

SQL-команда для создания индекса:

```
SET search_path TO lr3;  
  
CREATE INDEX idx_lesson_date ON lesson (lesson_date);  
ANALYZE lesson;
```

Результат после создания простого индекса:

	QUERY PLAN
	text
1	Seq Scan on lesson (cost=0.00..2.23 rows=53 width=36) (actual time=0.028..0.041 rows=52.00 loop...
2	Filter: (lesson_date >= (CURRENT_DATE - '1 mon'::interval))
3	Rows Removed by Filter: 18
4	Buffers: shared hit=1
5	Planning Time: 0.166 ms
6	Execution Time: 0.060 ms

Как можем заметить результаты одинаковые, так как строк мало и выгоднее использовать поиск по строкам чем индекс. Время примерно одинаково.

Формулировка запроса № 2 с составным индексом: Получить занятия конкретного преподавателя за последний год.

SQL-команда запроса № 2:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM lesson
WHERE teacher_id = 1
AND lesson_date >= CURRENT_DATE - INTERVAL '1 year';
```

Результат до создания составного индекса:

	QUERY PLAN
	text
1	Seq Scan on lesson (cost=0.00..2.40 rows=36 width=36) (actual time=0.046..0.058 rows=36.00 loop...
2	Filter: ((teacher_id = 1) AND (lesson_date >= (CURRENT_DATE - '1 year'::interval)))
3	Rows Removed by Filter: 34
4	Buffers: shared hit=1
5	Planning Time: 0.180 ms
6	Execution Time: 0.075 ms

SQL-команда для создания индекса:

```
SET search_path TO lr3;

CREATE INDEX idx_lesson_teacher_date ON lesson (teacher_id, lesson_date);
ANALYZE lesson;
```

Результат после создания составного индекса:

	QUERY PLAN	
	text	
1	Index Scan using idx_lesson_teacher_date on lesson (cost=0.15..12.87 rows=36 width=36) (actual time=0.019..0.024 rows=36.00 loo...	
2	Index Cond: ((teacher_id = 1) AND (lesson_date >= (CURRENT_DATE - '1 year'::interval)))	
3	Index Searches: 1	
4	Buffers: shared hit=2	
5	Planning Time: 0.113 ms	
6	Execution Time: 0.039 ms	

Можем увидеть, что во втором случае использовался индекс и время уменьшилось, хоть и не намного, потому что строк в таблицах не так уж и много.

SQL-команда для удаления индексов:

```
DROP INDEX idx_lesson_date;
DROP INDEX idx_lesson_teacher_date;
```

Выводы:

В ходе выполнения лабораторной работы были освоены продвинутые навыки работы с SQL в PostgreSQL. Были составлены сложные запросы с использованием объединения таблиц (JOIN), группировки (GROUP BY) и агрегатных функций. Созданы представления (VIEW) для скрытия сложной логики бизнес-запросов. Практически отработаны запросы на изменение данных (INSERT, UPDATE, DELETE) с применением подзапросов. Изучен механизм оптимизации запросов с помощью простых и составных индексов. Анализ планов выполнения (EXPLAIN ANALYZE) показал, что применение индексов меняет стратегию поиска с Sequential Scan на Index Scan, что ускоряет время обработки запроса.